

MEDS

Maktab-e-Digital Systems
Department of Electrical Engineering • UET Lahore

MODULE 4

SystemVerilog for Digital Design

Summer Training Programme 2026 • Cohort 4

Table of Contents

Table of Contents	2
1. Module Overview	4
2. Learning Objectives	4
Language Fundamentals	4
Simulation Workflow	4
Combinational Design	4
Sequential Design	4
Finite State Machines	5
Integration	5
3. How This Module Is Organised	5
3.1 Notation Used in This Document	5
4. SystemVerilog Fundamentals	6
4.0 What a SystemVerilog “Design” Actually Is	6
4.0.1 Your First Module, Line by Line	6
4.0.2 Instantiating a Module (Using It Inside Another Module)	7
4.1 Data Types	7
4.2 Operators	8
4.3 Combinational vs Sequential Timing	9
4.4 Timing Violations: Setup and Hold	10
4.5 Self-Check (No Submission Required)	11
5. Getting Started on EDA Playground	11
5.1 Setting Up a Project	11
5.1.1 Your First Simulation, Step by Step	12
5.1.2 Reading Simulator Error Messages	12
5.2 Minimal Design + Testbench Pair	12
5.2.1 \$display Format Specifiers	13
5.2.2 Reading a Waveform in EPWave	13
5.3 Self-Checking Testbenches	14
5.3.1 Exhaustive Testing With a for Loop	14
6. Combinational Design Practices	15
6.1 assign vs always	15
6.2 Blocking (=) vs Non-Blocking (<=) Assignment	17
6.3 The Event Queue (Why Order Sometimes Surprises You)	18
6.4 if-else vs case vs casez vs unique case	18
6.5 Tasks	19
7. Decoders, Encoders & One-Hot Logic	19
7.1 Decoders	19
7.2 One-Hot Encoding	20
7.3a Encoders	21
7.3 Tasks	22
8. Multiplexers & Shifters	22

8.1 MUX as a Universal Logic Element	22
8.1.1 Minterm Notation (Σm) — What It Means	24
8.2 Shifters and Barrel Shifters	24
8.3 Tasks	24
9. Flip-Flops, Registers & Reset	25
9.1 D and T Flip-Flops	25
9.2 Synchronous vs Asynchronous Reset	26
9.3 Tasks	27
10. Counters & Timing Applications	28
10.1 Frequency Dividers	28
10.1.1 The Clock-Division Math	28
10.2 Tasks	28
11. Finite State Machines	29
11.1 Moore vs Mealy	29
11.2 The Standard 3-Block FSM Coding Style	29
11.3 Worked Example: Detecting the Sequence “101” (Moore, with Overlap)	29
Step 1: The Specification	30
Step 2: The State Diagram (Described in Words and a Table)	30
Step 3: The SystemVerilog Implementation (3-Block Style)	30
Step 4: A Sketch of the Self-Checking Testbench	31
11.4 Excitation Tables: Deriving Flip-Flop Inputs From a State Sequence	32
11.5 Tasks	32
12. Synchronous FIFO Controller	33
12.1 Parameterized Modules (New Syntax You Will Need Here)	33
12.2 Objective	34
12.3 Specification	34
12.4 Module Interface	34
12.5 Suggested Design Approach	34
12.6 Tasks	35
13. Report Submission Format	35
14. Recommended Resources	37
Primary References	37
Quick Reference / Cheat Sheets	37
Coding Style: Why This Module Teaches the Rules It Does	37
Digital Logic Background	37
Practice	37
15. Glossary of Terms	38

1. Module Overview

Module 3 moves you from C and RISC-V into SystemVerilog — the language you will use to describe, verify, and reason about digital hardware for the rest of this programme. Where Module 2 and 3 taught you to think like a machine at the instruction level, Module 3 teaches you to think like the circuit itself: signals, timing, concurrency, and state.

This module is organised by topic rather than by day. Each section builds directly on the one before its fundamentals, then combinational design practice, then sequential design, then finite state machines and closes with an integration task that pulls every topic together into a single, realistic controller design.

■ DESIGN NOTE — Why Topic-wise, Not Day-wise?

Hardware design skill is cumulative, not scheduled. Some mentees will move through data types and operators quickly and spend longer on FSMs; others will need extra time debugging their first testbench. A topic-wise structure lets you set your own pace while still guaranteeing every concept is covered.

Simulation & Tooling: All design and simulation work in this module is done on EDA Playground. If you have a better grip on some other tool you can use that too but submission of code should be the link of EDA playground. (<https://edaplayground.com>) using the Synopsys VCS or Aldec Riviera-PRO simulator with SystemVerilog testbenches. No local toolchain installation is required.

Prerequisite: Module 1 (Linux/Git/Makefiles), Module 2 (C for Hardware Engineers) Module 3 (RISC-V Assembly) Completed.

Deliverable: At least one Task Report per topic (see Section 13), including all the deliverables specified in Section 13, must be submitted.

2. Learning Objectives

By the end of this module, every mentee should be able to:

Language Fundamentals

- Use SystemVerilog's 4-state and 2-state data types (logic, bit, reg, wire, int, byte) correctly and know when each applies
- Use arithmetic, relational, logical, bitwise, reduction, shift, and concatenation operators
- Explain setup time, hold time, and what a timing violation looks like on a waveform
- Distinguish combinational and sequential timing, and recognise unintentional latches

Simulation Workflow

- Set up a design + testbench pair on EDA Playground and run a simulation
- Write a self-checking testbench using initial blocks, \$display, \$monitor, and assertions
- Read a waveform (EPWave) to debug mismatches between expected and actual output

Combinational Design

- Use assign, always_comb, if-else, case, casez, and unique case correctly, and know when each is appropriate
- Explain blocking (=) vs non-blocking (<=) assignment and the SystemVerilog event/scheduling model
- Design decoders, encoders, one-hot logic, multiplexers, parity generators, and barrel shifters

Sequential Design

- Design D/T flip-flop based registers and counters with synchronous and asynchronous reset

- Explain the metastability problem and design a correct reset synchroniser
- Build frequency dividers and multi-digit counters (stopwatch-class designs)

Finite State Machines

- Distinguish Moore and Mealy machines and choose the right one for a given specification
- Design multi-input, multi-output FSMs and read-modify-write control FSMs
- Derive a state diagram and state table from a word specification, and implement it in SystemVerilog

Integration

- Combine combinational and sequential blocks, decoders, counters, and FSM control logic into one parameterized controller (the Section 12 FIFO task)

3. How This Module Is Organised

The module is split into the following topic sections. Work through them in order — later sections assume you are comfortable with everything before them.

#	Section	Core Topics
4	SystemVerilog Fundamentals	Data types, operators, timing violations
5	Getting Started on EDA Playground	Design + testbench workflow, running simulations
6	Combinational Design Practices	assign vs always, blocking/non-blocking, event queue, if-else vs case
7	Decoders, Encoders & One-Hot Logic	Decoders, priority encoders, leading-zero counters
8	Multiplexers & Shifters	MUX-based logic implementation, barrel shifters
9	Flip-Flops, Registers & Reset	D/T flip-flops, async vs sync reset, reset synchronisers
10	Counters & Timing Applications	Custom-sequence counters, frequency dividers, stopwatch
11	Finite State Machines	Moore/Mealy, multi I/O FSMs, read-modify-write, sequence detection
12	Integration Task	Parameterized synchronous FIFO controller
13	Report Submission Format	Applies to every section above
14	Recommended Resources	chipverify.com, vlsiverify.com, and more
15	Glossary of Terms	Quick reference for every technical term used in this module

3.1 Notation Used in This Document

This module assumes no prior SystemVerilog experience, so every construct is introduced with a worked example before you are asked to use it in a task. Four kinds of highlighted boxes appear throughout; know what each one means before you start reading:

■ KEY CONCEPT — Example Title

A box like this one explains a concept you must understand before moving on — read it fully, it is not optional background.

■ DESIGN NOTE — Example Title

A purple box like this one connects a topic to how it is actually used in real hardware design, or previews where you will reuse the idea later in the module.

■ COMMON PITFALL — Example Title

A red box like this one flags a mistake almost every beginner makes with this construct, and how to avoid it.

TASK N: Example Title

A blue box like this one is graded work. Every task box has a number; task numbers run continuously across the whole module (Task 1, Task 2, ...) so you and your mentor can refer to “Task N” randomly.

Plain paragraphs and grey code blocks (like the ones you already saw in Module 2 and 3) carry the core teaching content and worked examples — read the code blocks line by line, do not skim them. A short glossary of every technical term used in this module is provided in Section 15 if you forget what a word means partway through.

4. SystemVerilog Fundamentals

Goal: Build a solid vocabulary of module syntax, data types, operators, and timing concepts before writing a single line of RTL. This section assumes zero prior hardware description language experience — if you have never written a line of Verilog or SystemVerilog before, start here and read every worked example before continuing. There are no graded tasks in this section, but you must be able to answer the self-check questions at the end before moving to Section 5.

4.0 What a SystemVerilog “Design” Actually Is

Unlike a C program, which describes a sequence of instructions to execute, a SystemVerilog design describes a piece of hardware — physical wires, gates, and flip-flops that all exist and operate at the same time (concurrently), not one after another. This single idea is the biggest mental shift for anyone coming from a software background, and it is why the rest of this section spends so much time on data types and timing before touching any logic design.

4.0.1 Your First Module, Line by Line

A SystemVerilog design is built out of modules. A module is a named, reusable block of hardware with a defined set of input and output connections (its ports). Here is the smallest complete module you can write, annotated line by line:

```
module my_and_gate (    // 'module' begins a hardware block named my_and_gate
    input  logic a,    // a 1-bit input port named a
    input  logic b,    // a 1-bit input port named b
    output logic y      // a 1-bit output port named y
);                    // semicolon closes the port list

    assign y = a & b; // the ONE line of actual hardware: y is wired to a AND b

endmodule              // 'endmodule' closes the block – no semicolon here
```

- `module <name> ( <port list> ); ... endmodule` is the basic skeleton of every design file you will write in this module. Every module you write must end with an `endmodule`, and an `endmodule` never has a semicolon after it.
- Each port is declared as input or output (there is also `inout` for bidirectional signals, which you will not need in this module), followed by a type (almost always `logic` here) and a name.
- Inside the module body, `assign y = a & b;` is a continuous assignment: it says “y is permanently, continuously wired to the result of a AND b”. This is not a one-time computation the way a C assignment is — if a or b ever

changes, y updates immediately and automatically, because it describes a physical wire, not a step in a program.

4.0.2 Instantiating a Module (Using It Inside Another Module)

Once a module is written, it becomes a reusable building block: you can place (instantiate) it inside another module, the same way you would place a physical chip on a breadboard and wire its pins. This is how every design in this module is built: small, individually-tested pieces are wired together into larger ones.

```
module two_gates (
    input  logic a, b, c,
    output logic y
);
    logic ab_and; // an internal wire, not a port – only used inside this module

    // Named port connection: .port_name(signal_connected_to_it)
    // This is the ONLY style used in this module – it is far less error-prone
    // than positional connection, especially once a module has many ports.
    my_and_gate u_and1 (.a(a), .b(b), .y(ab_and));
    my_and_gate u_and2 (.a(ab_and), .b(c), .y(y));
endmodule
```

■ COMMON PITFALL — Named Port Connection Is Mandatory Style in This Module

`my_and_gate u_and1 (.a(a), .b(b), .y(ab_and));` explicitly says which local signal connects to which port of the instance, by name. The alternative — positional connection, e.g. `my_and_gate u_and1(a, b, ab_and);` — relies entirely on getting the port order right and is a leading cause of silent, hard-to-debug wiring bugs. Every instantiation in your submitted work must use a named connection.

4.1 Data Types

SystemVerilog inherits Verilog's 4-state types (which can hold 0, 1, X, or Z) and adds 2-state types borrowed from software languages. Hardware engineers default to 4-state types because X and Z carry real design information that a 2-state type would silently hide.

Type	States	Typical Use
logic	4 (0,1,X,Z)	Default type for signals in RTL; replaces reg/wire in most contexts
wire	4	Continuous-assignment nets driven by assign or a module port
reg	4	Legacy procedural storage (Verilog-2001); prefer logic in new code
bit	2 (0,1)	Testbench variables where X/Z tracking is not needed
byte, shortint, longint	2	Fixed-width testbench data (8/16/64-bit)
enum	Depends (later explained)	Named states for FSMs; always prefer over raw bit patterns

What do the four states actually mean? 0 and 1 are the logic levels you already expect. The other two carry design information that has no equivalent in a software language:

- X (unknown): the simulator genuinely does not know what this signal's value is — either because it has not been driven yet (e.g. a flip-flop before its first reset) or because two different values are being driven onto it in conflict. An X reaching your testbench output is almost always a bug and must never be ignored.

- Z (high-impedance): the signal is not being driven by anything right now — it is electrically floating, as on a shared/tri-state bus when no device is currently driving it. You will not need to produce Z outputs in this module's tasks, but you will see it if you forget to drive a signal.

Int	Integer
Available only in system Verilog	Available in both Verilog and System Verilog
Int is a 32-bit variable which is made of bit data type.	Int is a 32-bit variable which is made of reg data type
Can store only 2-state values	Can store 4-state values
This is not synthesizable	This is synthesizable as it represents a register of 32 flip-flops.

Declaring a vector (multi-bit signal) uses the [MSB:0] range syntax. The following module shows several declarations and prints their values with \$display, so you can see the 4-state behaviour directly in the simulation log:

```
module data_types_demo;
    logic [3:0] a;      // 4-bit vector, 4-state
    bit  [3:0] b;      // 4-bit vector, 2-state
    logic      c;      // single bit, 4-state

    initial begin
        $display("Before any assignment:");
        $display("  a (logic) = %b", a);    // prints xxxx - 4-state defaults to X
        $display("  b (bit)   = %b", b);    // prints 0000 - 2-state defaults to 0

        a = 4'b1010;
        c = 1'bz;
        $display("After assignment:");
        $display("  a = %b (decimal %0d)", a, a);
        $display("  c = %b", c);           // prints z
    end
endmodule
```

■ COMMON PITFALL — An Uninitialized logic Signal Is X, Not 0

Coming from C, it is natural to assume an unused variable defaults to 0. A 4-state logic signal in SystemVerilog defaults to X until something drives it. This is a deliberate feature: if your testbench ever displays an X where it expects a real value, that X is telling you an unknown or an uninitialized logic state — treat it as a bug report, not a display glitch.

■ COMMON PITFALL — logic Is Not reg or wire in Disguise

logic can be driven by exactly one source (like reg or wire), so it cannot model a true multi-driver net (e.g. a tri-state bus); for that you still need wire. In this module every internal signal and port should default to logic unless you specifically need a multi-driver net.

4.2 Operators

Category	Operators	Notes
Arithmetic	+ - * / % **	Width follows the widest operand; watch for unintended sign extension

Category	Operators	Notes
Relational	< <= > >=	Result is 1-bit; unsigned comparison unless operands are declared signed
Equality	== != === !==	=== / !== are 4-state case-equality; used in testbenches, never in synthesizable RTL
Logical	&& !	a && b, Returns true or false based on value of both expressions
Bitwise	& ^ ~ / ~^ ~	Operate bit-by-bit across the full vector width
Reduction	& ^ (unary, one operand)	Collapse a vector to 1 bit, e.g. &data is 1 only if every bit is 1
Shift	<< >> <<< >>>	<<< / >>> are arithmetic shifts and sign-extend for signed operands
Concatenation	{a,b}	Build wider vectors
Replication	{4{a}}	replication repeats a pattern N times
Conditional	cond ? true : false	A pure combinational mux in one expression

■ KEY CONCEPT — Reduction vs Bitwise: Same Symbol, Different Meaning

a & b (two operands) is a bitwise AND producing a vector. &a (one operand, prefix) is a reduction AND producing a single bit. This distinction trips up almost every beginner at least once, so read code carefully.

A short worked example makes the difference between bitwise and reduction operators concrete. Trace through what each \$display line below will print before checking the comment:

```

module operator_demo;
    logic [3:0] x = 4'b1101;
    logic [3:0] y = 4'b1001;

    initial begin
        $display("x & y   = %b", x & y);      // bitwise AND  -> 1001 (vector)
        $display("&x     = %b", &x);          // reduction AND-> 0    (not all 4 bits of x are 1)
        $display("|x     = %b", |x);          // reduction OR  -> 1    (at least one bit of x is 1)
        $display("^x     = %b", ^x);          // reduction XOR-> 1    (odd number of 1s in x:
three)
        $display("x == y  = %b", x == y);      // equality      -> 0    (1101 != 1001)
        $display("{x,y}   = %b", {x, y});      // concatenation-> 11011001 (8 bits)
        $display("{2{x}}  = %b", {2{x}});      // replication  -> 11011101 (x repeated twice)
    end
endmodule

```

4.3 Combinational vs Sequential Timing

Before setup and hold time can make sense, you need to be clear about the two fundamentally different kinds of timing behaviour a circuit can have:

- **Combinational timing:** the output changes some small propagation delay after the input changes, with no clock involved at all. An AND gate, a multiplexer, or an adder built only from assign / always_comb has this behaviour — change an input, and (after the gate delays settle) the output follows.
- **Sequential timing:** the output only changes on an active clock edge (posedge clk, in almost every design in this module). A flip-flop can sit with a completely stable output for as long as you like, and it only updates at

the instant the clock ticks. This is what lets a circuit “remember” a value — something pure combinational logic can never do.

4.4 Timing Violations: Setup and Hold

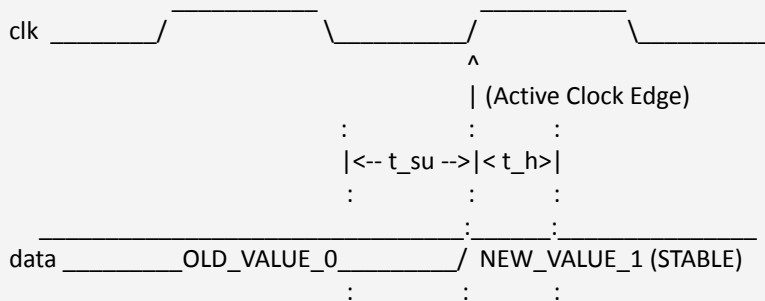
Every flip-flop needs its data input to be stable for a window around the active clock edge. Violating that window does not just risk a wrong value; it risks metastability, where the flop's output hovers between 0 and 1 for an unbounded time before settling.

Term	Definition	Violation Symptom
Setup time (t_{su})	Data must be stable this long BEFORE the clock edge	Data changes too close before the edge; flop may sample the old or new value unpredictably
Hold time (t_h)	Data must remain stable this long AFTER the clock edge	Data changes too soon after the edge; flop may sample the wrong value even though setup was met
Metastability	Output voltage sits between logic 0 and 1	Downstream logic sees a random 0/1 value that can differ between fanout paths

A timing diagram is normally drawn as waveforms, but the same information can be read directly off a text timeline. Read the diagram below left to right — time increases to the right, and the vertical bar | marks the active (rising) clock edge:

GOOD CASE — Setup and Hold Both Satisfied

In a good system, the data must settle to its new value at least t_{su} (setup time) before the clock edge, and maintain that value for at least t_h (hold time) after the clock edge.



Setup Time (t_{su}) Compliance: The signal transition from 0 to 1 occurs well outside the setup window. The data is valid and stable for the entire duration of t_{su} leading up to the clock edge.

Hold Time (t_h) Compliance: The data signal remains securely latched at logic level 1 throughout the post-edge hold window duration.

Result: The flip-flop successfully and reliably captures the stable NEW_VALUE_1 without any functional ambiguity or propagation delays.

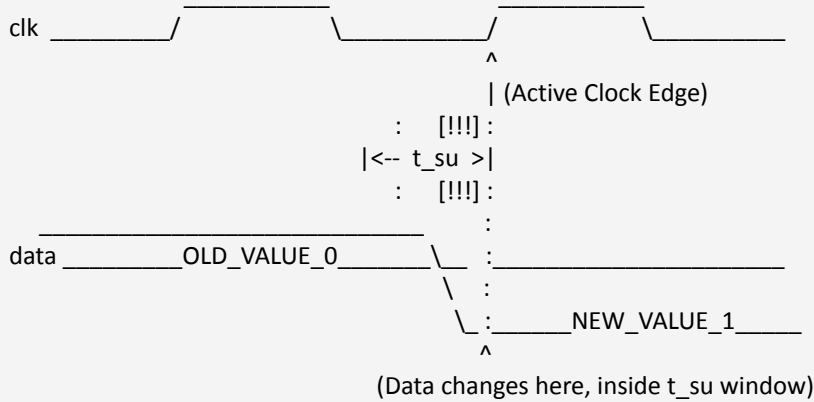
BAD CASE — Setup Violation

If the data changes state too close to the active clock edge—specifically inside the critical setup window—the hardware cannot guarantee which state is captured.

The Violation: The data signal begins its transition from 0 to 1 inside the critical setup time (t_{su}) window.

Result: The flip-flop’s internal master stage doesn’t have enough time to resolve the voltage level before the clock edge closes the sampling window.

Issue: The output is now undefined. The flip-flop may capture the OLD_VALUE_0, it may capture the NEW_VALUE_1, or its internal transistors may balance precariously in between, causing the output to go metastable (briefly oscillating or hovering at an invalid voltage level).



■ DESIGN NOTE — Where This Reappears In the Module

You will see the practical side of this in Section 9 (Reset Synchroniser), where deasserting an asynchronous reset at the wrong moment is exactly a setup/hold violation on the reset-recovery path of a flip-flop. Keep the “bad case” diagram above in mind — you will redraw a version of it yourself in that task.

4.5 Self-Check (No Submission Required)

Answer these for yourself before moving on to Section 5. If any answer feels shaky, re-read the relevant subsection above.

- What is the difference in behaviour between == and === when one operand contains an X?
- Why does logic replace both reg and wire in most modern RTL, and when would you still need wire?
- If a flop's D input changes 0.2 ns before the clock edge and t_{su} = 0.5 ns, what happens?
- What does the reduction operator ^data compute, and what physical circuit does it correspond to?
- What is the fundamental difference between combinational and sequential timing, in your own words?
- In the my_and_gate / two_gates example (Section 4.0), why is named port connection (.a(a), .b(b)) safer than positional connection (a, b, ab_and)?

5. Getting Started on EDA Playground

EDA Playground (<https://edaplayground.com>) is a free, browser-based environment that lets you write a design file and a testbench, pick a simulator, and see waveform and text results without installing anything. Every task in this module, including the Section 12 FIFO controller, is built and simulated here.

5.1 Setting Up a Project

- Create a free account at edaplayground.com so your playgrounds are saved and shareable via a link.
- On the left pane, select Language: SystemVerilog and a Tool such as Synopsys VCS or Aldec Riviera-PRO.
- Tick “Open EPWave after run” so a waveform viewer opens automatically after simulation.
- The left editor (design.sv) holds your DUT (device under test); the right editor (testbench.sv) holds your testbench, which instantiates the DUT.

5.1.1 Your First Simulation, Step by Step

If you have never run a simulation before, follow these steps exactly once, with the example in Section 5.2, before attempting Task 1.

- Step 1: Open edaplayground.com and log in (or continue as guest — but a free account is strongly recommended so your work is saved).
- Step 2: In the left-hand settings panel, set Language / Testbench to SystemVerilog.
- Step 3: Set Tools & Simulators to Synopsys VCS (or Aldec Riviera-PRO if VCS is unavailable in your account).
- Step 4: Tick the “Open EPWave after run” checkbox so a waveform opens automatically.
- Step 5: Paste your design code (the module you are building) into the left editor pane, labelled design.sv.
- Step 6: Paste your testbench code (which instantiates and drives your design) into the right editor pane, labelled testbench.sv.
- Step 7: Click the green Run button. The bottom panel will show compile and simulation log messages.
- Step 8: Read the log from the top. A syntax error will name a file and line number and stop the run before any \$display output appears — fix it and re-run. If the log instead shows your \$display output, the simulation ran; check the printed values against what you expected.
- Step 9: If EPWave opened, add your signals to the waveform view (drag them from the signal list on the left into the waveform pane) and confirm the value changes match your \$display output.

5.1.2 Reading Simulator Error Messages

Almost every first-time SystemVerilog error falls into one of three categories. Learning to recognise which one you are looking at saves enormous debugging time:

Error Category	When It Appears	Typical Cause
Compile / syntax error	Immediately, before simulation starts	Missing semicolon, unmatched begin/end, misspelled keyword, mismatched port name in an instantiation
Elaboration error	After compile, before simulation runs	Port width mismatch between a module and its instantiation, undeclared module name, wrong number of ports connected
Runtime / simulation warning	While the simulation is running, printed in-line with your \$display output	An X propagating somewhere unexpected, a testbench assertion firing, a \$finish never being reached (simulation times out)

5.2 Minimal Design + Testbench Pair

Every task report in this module must link to a playground built on this pattern:

```
// ===== design.sv =====
module and_gate (
    input  logic a, b,
    output logic y
);
    assign y = a & b;
endmodule

// ===== testbench.sv =====
```

```

module tb;
  logic a, b, y;

  and_gate dut (.a(a), .b(b), .y(y));

  initial begin
    // This is a must to include to open the waveform
    $dumpfile("dump.vcd");
    $dumpvars(0, tb);

    a = 0; b = 0; #10;
    $display("a=%0b b=%0b y=%0b", a, b, y);
    a = 1; b = 0; #10;
    $display("a=%0b b=%0b y=%0b", a, b, y);
    a = 1; b = 1; #10;
    $display("a=%0b b=%0b y=%0b", a, b, y);

    $finish;
  end
endmodule

```

5.2.1 \$display Format Specifiers

\$display works much like C's printf, but with formats suited to hardware values. You will use these constantly, in every testbench you write for the rest of the module:

Specifier	Prints	Example
%b	Binary, showing every bit including leading 0/X/Z	\$display("%b", 4'b0101) → 0101
%0b	Binary, without leading zero padding	\$display("%0b", 4'b0101) → 101
%d / %0d	Decimal (unsigned unless the signal is declared signed)	\$display("%0d", 4'b1010) → 10
%h / %0h	Hexadecimal	\$display("%0h", 8'hA5) → a5
%t	Simulation time (use with \$time)	\$display("time=%0t", \$time)
%s	String	\$display("result=%s", pass_str)
%m	The hierarchical instance path of the current module	Useful when a \$display is inside a reusable module instantiated many times

■ KEY CONCEPT — \$display vs \$monitor vs \$strobe

\$display(...) prints a message once, exactly where it appears in your code. \$monitor(...) is enabled once and then automatically prints a new line whenever one of the signals in its argument list changes, making it useful for continuously watching signals during simulation. \$strobe(...) also prints once like \$display(...), but it waits until the end of the current simulation time step before printing, ensuring it shows the final settled values after all updates for that time step have completed.

5.2.2 Reading a Waveform in EPWave

A waveform is the same information as a \$display log, but shown visually against a time axis — and it is often far faster for spotting a bug in a sequential design. Once EPWave opens after a run:

- The left-hand panel lists every signal in your testbench hierarchy; drag the ones you care about (clk, reset, and your DUT's inputs/outputs) into the waveform pane.
- The horizontal axis is simulation time; use the zoom controls (or scroll) to expand a region where you expect a transition.
- Right-click a signal to change its radix (display base) between binary, decimal, and hex — use binary for 1-bit control signals and hex/decimal for wide data buses.
- A thin vertical cursor line can be dragged across the waveform; the value of every visible signal at that exact time is shown in the value column, which is the fastest way to check “what was rd_data at time 45ns?”.

5.3 Self-Checking Testbenches

A testbench that only prints values still requires a human to check the log by eye. From this point on, every testbench you write should compare the DUT's output against an expected value and report PASS/FAIL automatically.

```
task automatic check(input logic actual, expected, string name);
    if (actual !== expected)
        $display("[FAIL] %s: expected=%0b actual=%0b at time=%0t", name, expected, actual,
$time);
    else
        $display("[PASS] %s: value=%0b at time=%0t", name, actual, $time);
endtask
```

■ KEY CONCEPT — Why !== Instead of != in a Checker

!== is the 4-state inequality check; it also flags X or Z on the DUT output, which a plain != would silently mistreat. A checker that cannot detect an X on the output is not really self-checking.

5.3.1 Exhaustive Testing With a for Loop

Many tasks in this module ask you to “test all N input combinations”. Writing out 8 or 16 stimulus lines by hand is slow and easy to get wrong; a for loop that sweeps every combination and computes its own expected value is faster to write and far more trustworthy. This pattern, built on the check() task above, is what you should reuse for every exhaustive-test requirement in Sections 6–8:

```
module tb;
    logic [2:0] abc;      // 3-bit input, will sweep 3'b000 through 3'b111
    logic      a, b, c, y;

    my_function_dut dut (.a(a), .b(b), .c(c), .y(y));

    task automatic check(input logic actual, expected, string name);
        if (actual !== expected)
            $display("[FAIL] %s: expected=%0b actual=%0b", name, expected, actual);
        else
            $display("[PASS] %s: value=%0b", name, actual);
    endtask

    initial begin
        for (abc = 3'b000; abc <= 3'b111; abc = abc + 1) begin
            {a, b, c} = abc;          // unpack the loop variable onto the 3 inputs
            #1;                       // let combinational logic settle
            check(y, expected_fn(a, b, c), $sformatf("abc=%03b", abc));
        end
        $finish;
    end

    // A plain SystemVerilog function computing the SAME truth table by hand,
```

```
// used purely as an independent reference model to check the DUT against.
function automatic logic expected_fn(input logic a, b, c);
    return (a & ~b) | (b & c); // replace with YOUR function's truth table
endfunction
endmodule
```

■ KEY CONCEPT — Why a Separate expected_fn() Function Matters

The reference model in expected_fn() must be written independently from your DUT's implementation — usually as a direct translation of the truth table or specification, not copied from your RTL. If you accidentally make the same mistake in both the DUT and the checker, the testbench will falsely report [PASS] on a broken design. This is the single most common reason a “self-checking” testbench misses a real bug.

TASK 1: Hello Gate

Reproduce the AND-gate design/testbench pair above on EDA Playground exactly as shown, run it, and confirm the expected output appears in the log.

- Extend the testbench to also test the OR and XOR of the same two inputs.
- Open EPWave and confirm the a, b, y waveforms match the truth table.

TASK 2: Convert to a Self-Checking Testbench

Rewrite the testbench from Task 1 using the check() task pattern above so each stimulus vector automatically prints PASS or FAIL.

- Intentionally introduce one wrong expected value to confirm your checker prints [FAIL] when it should, then remove the bug before final submission.

“ See the Report Submission Format (Section 13) for how work on this topic must be documented and submitted. ”

6. Combinational Design Practices

Goal: Learn the three ways SystemVerilog lets you describe combinational logic (assign, always_comb / always @*, and always with if-else or case), understand blocking vs non-blocking assignment, and understand the simulation event queue well enough to explain why a testbench sometimes samples the “old” value of a signal.

6.1 assign vs always

Construct	Style	When to Use
assign lhs = expr;	Continuous, dataflow	Simple combinational expressions with one clear equation; wires only
always_comb begin ... end	Procedural, combinational	Multi-line combinational logic, if-else / case blocks; SystemVerilog auto-detects the sensitivity list and flags incomplete assignments
always @(*) begin ... end	Procedural, combinational (legacy)	Same as always_comb but relies on the tool to infer sensitivity correctly; prefer always_comb in new code
always_ff @(posedge clk) begin ... end	Procedural, sequential	Anything that must be remembered between clock edges — never combinational logic

■ COMMON PITFALL — Incomplete if / case Creates an Unintended Latch

Forgetting to assign a signal on every branch of an `always_comb` block. If a signal is left unassigned in any path (such as an if without an else or a case without a default), the synthesis tool infers a latch to preserve the previous value. In most cases, this is unintended and should be treated as a bug.

Worked example — a 2-to-1 multiplexer written both ways, so you can see they describe exactly the same hardware:

```
module mux2_assign (
    input logic sel, a, b,
    output logic y
);
    // Style 1: Continuous assignment using the ternary operator
    assign y = sel ? b : a;
endmodule

module mux2_always (
    input logic sel, a, b,
    output logic y
);
    // Style 2: always_comb with if-else – useful when the logic
    // becomes more complex than a single expression
    always_comb begin
        if (sel)
            y = b;
        else
            y = a;      // Every branch assigns y: no latch is inferred
    end
endmodule

module mux2_case (
    input logic sel, a, b,
    output logic y
);
    // Style 3: always_comb with a case statement – often preferred
    // when selecting between multiple cases
    always_comb begin
        case (sel)
            1'b0: y = a;
            1'b1: y = b;
            default: y = a; // Defensive coding style
        endcase
    end
endmodule
```

■ COMMON PITFALL — The Missing-else That Creates a Latch, Shown Directly

`always_comb begin if (sel) y = b; end` — with no else — leaves `y` unassigned whenever `sel` is 0. Because `always_comb` cannot leave `y` undriven, the tool infers a latch that holds `y`'s previous value until `sel` becomes 1 again. This is very rarely what you want; the fix is always to add the else (or, for a case statement, a default: branch) so every input combination is covered.

6.2 Blocking (=) vs Non-Blocking (<=) Assignment

Assignment	Executes	Correct Usage
Blocking (=)	Immediately, in program order, before the next statement runs	Combinational logic (always_comb) and testbench sequencing
Non-blocking (<=)	Scheduled; all right-hand sides are read first, then all left-hand sides update together at the end of the time step	Sequential logic (always_ff)

■ COMMON PITFALL — Mixing = and <= in the Same always_ff Block

Using blocking assignment inside a clocked always_ff block can create simulation-vs-synthesis mismatches, because the simulator updates signals immediately while real flip-flops update on the clock edge together. Rule: always_comb uses only =, always_ff uses only <=.

The clearest way to see why this rule exists is to build a 2-stage shift register (two D flip-flops in series) both the wrong way and the right way, and trace what happens on one clock edge where in = 1 and both flops currently hold 0.

```
// WRONG: blocking assignment inside always_ff
module shift2_broken (
    input logic clk, in,
    output logic q1, q2
);
    always_ff @(posedge clk) begin
        q1 = in; // executes FIRST, immediately: q1 becomes 1 right away
        q2 = q1; // executes SECOND, reading the NEW q1: q2 ALSO becomes 1
    end
    // BUG: both flops now show 1 on the SAME edge. In real hardware this is
    // impossible – q2 should still be showing the OLD value of q1 this cycle.
endmodule

// CORRECT: non-blocking assignment inside always_ff
module shift2_correct (
    input logic clk, in,
    output logic q1, q2
);
    always_ff @(posedge clk) begin
        q1 <= in; // RHS (in) is read now, but q1's update is SCHEDULED
        q2 <= q1; // RHS (q1) is read using its OLD (pre-edge) value,
                // because the scheduled update above has not happened yet
    end
    // CORRECT: q2 gets the OLD q1, and q1 gets 'in', both updating together
    // at the end of the time step – exactly like two real flip-flops.
endmodule
```

■ KEY CONCEPT — Reading the Trace: Why <= Gets the “Old” Value

For non-blocking assignment, SystemVerilog evaluates every right-hand side in the block first (using the values signals had before the clock edge), and only after ALL right-hand sides are evaluated does it apply the left-hand-side updates, together, in one step. That is exactly what a bank of real flip-flops does: every flop samples its D input at the same instant. Blocking assignment, by contrast, updates q1 immediately, so the very next line sees the brand-new value instead of the old one — which is why shift2_broken loses one stage of delay.

6.3 The Event Queue (Why Order Sometimes Surprises You)

SystemVerilog simulation time is divided into time steps, and each time step is divided into ordered regions: Active (blocking assignments, RHS evaluation for non-blocking), NBA / Non-blocking Assign Update (LHS of `<=` actually updates), Observed (assertions), and Reactive/Postponed (`$strobe`, `$monitor`). This is why a testbench that reads a signal immediately after driving a clocked `always_ff` block with `#0` can see the pre-update value: the NBA update has not happened yet in that same time step.

■ KEY CONCEPT — Practical Rule of Thumb

In a testbench, sample DUT outputs slightly after the clock edge (e.g. `#1` after `posedge clk`, or on the `negedge`) rather than exactly at the edge, so you are guaranteed to see the settled, post-NBA-update value.

6.4 if-else vs case vs casez vs unique case

- if-else: best for priority logic where conditions are evaluated top-to-bottom and the first true branch wins.
- case: best for parallel, mutually-exclusive selection (e.g. opcode decode); all branches are logically equal priority.
- casez: like case but treats `?` (or `z`) as a don't-care bit in the case item, useful for priority encoders and instruction decoding with wildcard fields.
- unique case: instructs the tool to assume exactly one branch matches, enabling extra simulation warnings if that assumption is violated and enabling more efficient logic; use it whenever you can guarantee mutual exclusivity.

Worked example — a simple 2-bit ALU-select block using an ordinary case, immediately followed by a 4-input priority encoder using `casez`, so you can compare the two constructs side by side:

```
module alu_select_demo (
    input logic [1:0] op_sel,    // ordinary, mutually exclusive selection
    input logic [3:0] a, b,
    output logic [3:0] result
);
    always_comb begin
        case (op_sel)
            2'b00: result = a + b;
            2'b01: result = a - b;
            2'b10: result = a & b;
            2'b11: result = a | b;
            default: result = 4'b0000; // covers X/Z on op_sel – always include this
        endcase
    end
endmodule

module priority_encoder4_demo (
    input logic [3:0] req,      // one or more request lines may be high
    output logic [1:0] grant_idx,
    output logic valid
);
    always_comb begin
        valid = |req;           // reduction OR: 1 if ANY request line is high
        casez (req)
            // '?' means "don't care" – bit 3 (highest priority) checked first
            4'b1??? : grant_idx = 2'd3;
            4'b01?? : grant_idx = 2'd2;
            4'b001? : grant_idx = 2'd1;
            4'b0001 : grant_idx = 2'd0;
            default: grant_idx = 2'd0;
        endcase
    end
endmodule
```

```
end
endmodule
```

■ KEY CONCEPT — Reading casez: Highest Priority Line Wins

In `priority_encoder4_demo`, if `req = 4'b1010` (bits 3 and 1 both requesting), the FIRST matching casez item wins top-to-bottom: `4'b1???` matches (bit 3 is 1, the ?s don't care about the rest), so `grant_idx = 3`, even though bit 1 is also requesting. This top-to-bottom priority behaviour is why casez is the standard tool for priority encoders — exactly the pattern you will reuse for the Leading Zero Counter task in Section 7.

6.5 Tasks

TASK 3: Half Adder and Full Adder: assign vs always_comb

Implement a half adder (sum, carry from a, b) two different ways in the same file: once using only continuous assign statements, and once using an `always_comb` block with an if-else (or case) structure. Instantiate both and confirm identical outputs for all 4 input combinations.

- Extend to a full adder (a, b, cin → sum, cout) using structural composition of two half adders plus an OR gate.
- In your report, state in your own words why the assign version and the `always_comb` version synthesize to the same gates even though they are written differently.

TASK 4: NOR-Only Realization

Find a NOR-gate-only circuit realization of $f = (b + c + d)(a' + b + c)(a' + d)$.

- Show your Boolean algebra / De Morgan steps for converting the AND-OR expression into an all-NOR network.
- Implement f in SystemVerilog two ways: (1) a direct assign using the original expression, and (2) an assign built purely from the `~|` (NOR) reduction/bitwise pattern you derived.
- Testbench must sweep all 16 combinations of a, b, c, d and confirm both implementations match.

TASK 5: 4-bit Parity Generator

Create a parity generator module with a 4-bit input data and a 1-bit output parity. parity must be high (1) if the number of high bits in data is odd, and low (0) otherwise.

- Implement it using the reduction XOR operator in a single assign statement.
- Testbench must exhaustively check all 16 input combinations against a `$countones()`-based (or manually computed) expected value.

| “See the Report Submission Format (Section 13) for how work on this topic must be documented and submitted.”

7. Decoders, Encoders & One-Hot Logic

Goal: Understand decoders (one input code → many one-hot outputs) and encoders (many inputs → one binary code), and see why one-hot encoding is such a common pattern in FSM state registers and control logic.

7.1 Decoders

An n -to- 2^n decoder asserts exactly one of 2^n outputs based on an n -bit binary input. Decoders can be built hierarchically: a larger decoder is built from smaller decoders plus enable logic, exactly the way a memory address decoder is built from row/column sub-decoders in real hardware.

Concretely: a 2-to-4 decoder has a 2-bit input `sel` and a 4-bit output `y`, where exactly one bit of `y` is 1 (the bit at index `sel`) and the rest are 0 — unless an enable input `en` is low, in which case every output is forced to 0 regardless of `sel`. Here is the truth table, followed by a full worked implementation:

en	sel[1:0]	y[3:0]
0	xx	0000
1	00	0001
1	01	0010
1	10	0100
1	11	1000

```

module decoder2to4 (
    input logic en,
    input logic [1:0] sel,
    output logic [3:0] y
);
    always_comb begin
        if (!en)
            y = 4'b0000;          // disabled: every output forced low
        else begin
            case (sel)
                2'b00: y = 4'b0001;
                2'b01: y = 4'b0010;
                2'b10: y = 4'b0100;
                2'b11: y = 4'b1000;
                default: y = 4'b0000;
            endcase
        end
    end
endmodule

```

“Look closely at the **y** output above, for any valid **sel**, exactly one bit of **y** is 1 and all the rest are 0. This pattern one bit hot, everything else off — is called **one-hot**.”

■ DESIGN NOTE — Why Decoders Matter Beyond This Task

This is precisely the mechanism your computer uses to select one memory location out of millions from a binary address, and the mechanism you will use in Section 11 to translate an FSM's one-hot or binary state encoding into individual control signals for a datapath.

7.2 One-Hot Encoding

One-hot encoding represents each of N possible states or values using an N -bit vector in which exactly one bit is 1 at any time — the same pattern the 7.1 decoder produces. This is in contrast to binary encoding, which packs the same N values into just $\log_2 N$ bits.

■ KEY CONCEPT — Why One-Hot for FSM States?

With one-hot encoding, each state's “active” signal is a single flip-flop, so next-state and output logic for that state is often just a few gates instead of a full binary decode. The tradeoff is more flip-flops (one per state instead of $\log_2(N)$);

for small-to-medium FSMs like the ones in this module, that tradeoff is almost always worth it for the speed and readability gain.

7.3a Encoders

A decoder turns a binary code into one-hot; an encoder does the reverse — it turns a one-hot input into a binary code. If you already understand the decoder in 7.1, an encoder is that same table read right-to-left.

Concretely: a 4-to-2 encoder takes a 4-bit one-hot input *in* and produces a 2-bit output *y* equal to the index of the bit that's high.

```
module encoder4to2 (
    input  logic [3:0] in,
    output logic [1:0] y
);
    always_comb begin
        case (in)
            4'b0001: y = 2'b00;
            4'b0010: y = 2'b01;
            4'b0100: y = 2'b10;
            4'b1000: y = 2'b11;
            default: y = 2'bxx; // in wasn't one-hot: undefined which index to report
        endcase
    end
endmodule
```

■ LIMITATION — What Happens With Multiple or Zero Inputs?

The encoder above only behaves sensibly when *exactly one* input bit is high. What should *y* be if *in* = 4'b0110 (two bits high) or *in* = 4'b0000 (none high)? The plain encoder has no good answer — it just falls through to the default case and outputs garbage, with no way to signal that the result isn't meaningful. In practice, inputs to an encoder are often independent *requests* from multiple sources (interrupt lines, button presses, grant requests), so there's no guarantee only one is active at a time. Closing this gap is exactly what the **priority encoder** in 7.3b is for.

7.3b Priority Encoders

A priority encoder resolves the plain encoder's problem above: when multiple inputs are asserted at once, it outputs the index of the highest-priority one (by convention, the highest-indexed bit wins).

in[3:0]	y[1:0]	meaning
1???	11	bit 3 wins regardless of bits 2:0
01??	10	bit 2 wins if bit 3 is low
001?	01	bit 1 wins if bits 3:2 are low
0001	00	only bit 0 asserted
0000	xx	nothing asserted — output not meaningful

As we saw earlier with `casez`, the `?` marks are don't-cares — row 1 matches 1000, 1001, 1010, ... 1111, since bit 3 always wins the tie regardless of the lower bits.

7.3 Tasks

■ DESIGN NOTE — How to Approach Task 6

A 3-to-8 decoder splits naturally into two 2-to-4 decoders: use the top address bit (`sel[2]`) as the enable of one 2-to-4 decoder and the inverted top bit as the enable of the other, feeding the remaining two bits (`sel[1:0]`) to both instances' sel input. Only one of the two 2-to-4 decoders will ever be enabled at a time, so their 4-bit outputs can simply be concatenated into the full 8-bit result.

TASK 6: 3-to-8 Decoder from 2-to-4 Decoders

Implement a 3-to-8 decoder using two instances of a 2-to-4 decoder module plus additional enable/gate logic (do not just write `assign y = 8'b1 << sel`).

- First design and verify the 2-to-4 decoder (with an enable input) as a standalone module.
- Then instantiate it twice and add the glue logic that produces the full 3-to-8 decode.
- Testbench must exhaustively check all 8 select values with enable asserted, plus confirm all outputs are 0 when enable is deasserted.

TASK 7: 32-bit Leading Zero Counter (LZC)

Implement a 32-bit Leading Zero Counter that returns the number of leading zeros in a 32-bit input (e.g. `32'h00000001` → 31, `32'h80000000` → 0, `32'h00000000` → 32).

- First implement a simple version using a priority-encoder-style `casez` or a for-loop inside `always_comb`.
- Then implement (or at least describe in your DESIGN section) a tree-structured version that finds leading zeros in 2-bit, then 4-bit, then 8-bit groups in parallel, reducing logic-level depth compared to the linear priority-encoder version.
- In your report, explain in your own words how many logic levels each version has in its critical path and why the tree version is faster for large bit widths.
- Testbench must check at least: all-zero input, single bit set at each of a few positions (0, 15, 31), and a fully random sweep of 50+ vectors compared against a `$clog2`/for-loop reference model.

“ See the Report Submission Format (Section 13) for how work on this topic must be documented and submitted. ”

8. Multiplexers & Shifters

Goal: Practice implementing arbitrary Boolean functions using multiplexers as universal logic elements, then extend that skill to build fixed shifters and a mux-based barrel shifter — a building block you will see again in ALU and datapath design.

8.1 MUX as a Universal Logic Element

A multiplexer (MUX) selects one of several data inputs to route to its output, based on a select input. The simplest case, a 2:1 mux, has 2 data inputs (`d0`, `d1`), a 1-bit select `sel`, and one output `y`, defined by `y = sel ? d1 : d0`. Larger multiplexers follow the same pattern with more select bits: a 4:1 mux has 4 data inputs and a 2-bit select, an 8:1 mux has 8 data inputs and a 3-bit select, and so on.

```
module mux4to1 (
    input  logic [1:0] sel,
```

```

input logic      d0, d1, d2, d3,
output logic     y
);
always_comb begin
    case (sel)
        2'b00: y = d0;
        2'b01: y = d1;
        2'b10: y = d2;
        2'b11: y = d3;
        default: y = 1'b0;
    endcase
end
endmodule

```

Any Boolean function of n variables can be implemented with a $2^n-1:1$ multiplexer by wiring $n-1$ of the variables to the select lines and connecting each data input to whatever the remaining variable's function reduces to for that select combination (a constant 0, a constant 1, the variable itself, or its complement). This is a standard technique for realizing a given truth table or minterm list directly in hardware without deriving a minimized SOP/POS expression first.

Worked example: implement $F(a,b) = a \text{ XOR } b$ (2 variables) on a 4:1 mux, wired the standard way — both variables a, b on the select lines, and the (empty, since there are no variables left) remaining function reduces to a constant at each data input:

a b (=sel)	F = a XOR b	MUX data input driven with
00	0	d0 = 1'b0
01	1	d1 = 1'b1
10	1	d2 = 1'b1
11	0	d3 = 1'b0

```

module xor_via_mux (
    input logic a, b,
    output logic y
);
    mux4to1 u_mux (
        .sel({a, b}), // concatenation: sel[1]=a, sel[0]=b
        .d0(1'b0), .d1(1'b1), .d2(1'b1), .d3(1'b0),
        .y(y)
    );
endmodule

```

■ KEY CONCEPT — The General Recipe You Will Reuse in the Task Below

1) Write the truth table of your function. 2) Choose which variables drive the select lines (for an n -variable function on a $2^k-1:1$ mux, k of the variables go on select). 3) For every select combination, read off what the output must equal in terms of the remaining variable(s) — it will always reduce to one of exactly four things: constant 0, constant 1, the remaining variable itself, or its complement. 4) Wire each data input to whichever of those four applies. This is exactly the method the task below asks you to apply and document.

8.1.1 Minterm Notation (Σm) — What It Means

A minterm of an n -variable Boolean function is one row of its truth table, numbered by treating that row's input combination as a binary number. For 3 variables a, b, c , minterm 5 is the row where $(a,b,c) = (1,0,1)$, because binary 101 = decimal 5. Writing $F = \Sigma m(1,2,3,6,7)$ is shorthand for “ F is 1 exactly on minterms 1, 2, 3, 6, and 7, and 0 on every other row” — it fully specifies the truth table without drawing the table out.

Minterm #	a b c	In $\Sigma m(1,2,3,6,7)$?	F
0	000	no	0
1	001	yes	1
2	010	yes	1
3	011	yes	1
4	100	no	0
5	101	no	0
6	110	yes	1
7	111	yes	1

■ KEY CONCEPT — You Do Not Need to Minimize the Boolean Expression for a MUX Implementation

The whole appeal of the MUX-realization technique in Section 8.1 is that it works directly from the minterm list (or truth table) — you never need to run Karnaugh-map minimization or derive a minimal SOP/POS expression first. That said, minimizing an expression is still useful for the tasks in Section 6 that ask for a written Boolean derivation (e.g. the NOR-only realization task), where standard Boolean algebra and De Morgan's laws are what you should use; a Karnaugh map is simply a visual shortcut for the same algebra and is optional here.

8.2 Shifters and Barrel Shifters

A fixed single-position shifter moves every bit one place and fills the vacated position with a constant or a carry-out bit. A barrel shifter generalises this to an arbitrary shift/rotate amount selected at run time, and is naturally implemented as a cascade of 2:1 muxes, one stage per power-of-two shift amount — which is why the task below asks you to design the single-position version first, as a warm-up, before extending it to a full barrel shifter.

8.3 Tasks

TASK 8: $F = \Sigma m(1,2,3,6,7)$ Using a 4x1 MUX

Implement $F = \Sigma m(1,2,3,6,7)$ (three variables a, b, c) using a 4x1 multiplexer.

- Draw the full truth table of F over a, b, c (Section 8.1.1 shows you how to expand the $\Sigma m(\dots)$ notation into a table).
- Show each step of the implementation: choosing which two variables drive the select lines, and deriving what constant or literal (0, 1, c , or c') must be wired to each of the 4 data inputs.
- Implement the resulting circuit in SystemVerilog using a 4x1 mux built from a case statement, and instantiate it with a, b as select and the derived data-input expressions.
- Testbench must check all 8 minterms against the given minterm list.

- Extension (include in the same report): repeat the exercise for a 3-bit majority function (output is 1 when 2 or more of a, b, c are 1), this time using only cascaded 2-to-1 multiplexers instead of a single 4:1 mux. Include the majority function's truth table and a schematic showing how the 2:1 muxes are cascaded.

TASK 9: 4-bit Barrel Shifter (Rotate) Using unique case

This task has two parts — design the simple fixed shifter first, then extend it to a full barrel shifter.

- Part A (single-position shift): design a circuit that shifts a 4-bit vector $W = w_3 w_2 w_1 w_0$ one bit position to the right when a control signal $\text{shift} = 1$, producing output $Y = y_3 y_2 y_1 y_0$ and a shifted-out bit k , such that if $\text{shift} = 1$: $y_3=0, y_2=w_3, y_1=w_2, y_0=w_1, k=w_0$; and if $\text{shift} = 0$: $Y=W, k=0$. Implement using a conditional (mux) expression per bit, not a case statement.
- Part B (barrel shifter): design a four-bit barrel shifter using multiplexers that rotates the input $W = w_3 w_2 w_1 w_0$ by 0, 1, 2, or 3 bit positions into output $Y = y_3 y_2 y_1 y_0$ as selected by a 2-bit control sel : $\text{sel}=00 \rightarrow Y=W$; $\text{sel}=01 \rightarrow y_3=w_0, y_2=w_3, y_1=w_2, y_0=w_1$; $\text{sel}=10 \rightarrow y_3=w_1, y_2=w_0, y_1=w_3, y_0=w_2$; $\text{sel}=11 \rightarrow y_3=w_2, y_2=w_1, y_1=w_0, y_0=w_3$.
- Part B must use a unique case statement for the sel decode. In your report, explain what extra guarantee unique case gives the simulator/synthesizer versus a plain case here.
- Testbench: cover $\text{shift}=0/1$ for Part A across several values of W ; cover all 4 values of sel for Part B against at least 3 different values of W each.

“ See the Report Submission Format (Section 13) for how work on this topic must be documented and submitted. ”

9. Flip-Flops, Registers & Reset

Goal: Move from combinational to sequential design. Understand D and T flip-flop behaviour, the difference between synchronous and asynchronous reset, and why an asynchronous reset needs a synchroniser on its deassertion edge.

9.1 D and T Flip-Flops

A flip-flop is the fundamental one-bit memory element of digital hardware: unlike a wire or a combinational gate, its output does not simply follow its input — it captures (“latches”) the input's value only at the active clock edge, and holds that value steady until the next active edge, no matter what the input does in between.

Flip-Flop	Behaviour	SystemVerilog Pattern
D Flip-Flop	Q takes the value of D on the active clock edge	<code>always_ff @(posedge clk) q <= d;</code>
T Flip-Flop	Q toggles on the active clock edge when $T=1$, holds when $T=0$	<code>always_ff @(posedge clk) q <= q ^ t;</code>

Here is a complete, minimal D flip-flop module, followed by a multi-bit register built from it (which is exactly what a real hardware register is: a bank of D flip-flops sharing one clock and one enable):

```

module d_flip_flop (
    input logic clk,
    input logic d,
    output logic q
);
    always_ff @(posedge clk) begin
        q <= d;    // captures d at every rising edge of clk; holds otherwise
    end
endmodule

module register_8bit (

```

```

input logic      clk,
input logic      en,      // load enable
input logic [7:0] d,
output logic [7:0] q
);
always_ff @(posedge clk) begin
    if (en)
        q <= d; // only capture a new value when en is asserted
    // no else needed: if en=0, q simply keeps its current value,
    // which is exactly what a real flip-flop does when not updated
end
endmodule

```

■ KEY CONCEPT — Why There Is No else in register_8bit — and Why That's Correct

This is the one place the “every branch must be assigned” rule from Section 6.1 does NOT apply. `always_comb` requires every branch to assign the signal because combinational logic has no memory of its own — an unassigned branch would need a latch to “remember” anything. `always_ff` already has memory built in (it is a flip-flop): when the if condition is false and there is no else, `q` correctly and simply keeps holding its last captured value. This is the normal, intended way to describe an enabled register.

9.2 Synchronous vs Asynchronous Reset

Reset Style	Sensitivity List	Behaviour
Synchronous	<code>always_ff @(posedge clk)</code>	Reset only takes effect on the next active clock edge; simpler timing closure, but the clock must be running for reset to work
Asynchronous	<code>always_ff @(posedge clk or negedge rst_n)</code>	Reset takes effect immediately, independent of the clock; needed when you cannot guarantee a clock edge arrives soon after power-up, but its deassertion edge can itself cause a timing violation if not synchronised

The two reset styles differ only in the `always_ff` sensitivity list and the if condition inside it — but that small difference changes exactly when reset takes effect:

```

module d_ff_sync_reset (
    input logic clk, rst_n, d,
    output logic q
);
    // Synchronous: rst_n is NOT in the sensitivity list.
    // Reset only clears q on the NEXT posedge clk after rst_n goes low.
    always_ff @(posedge clk) begin
        if (!rst_n)
            q <= 1'b0;
        else
            q <= d;
    end
endmodule

module d_ff_async_reset (
    input logic clk, rst_n, d,
    output logic q
);
    // Asynchronous: rst_n IS in the sensitivity list (negedge rst_n).
    // Reset clears q IMMEDIATELY when rst_n goes low, with no clock needed.
    always_ff @(posedge clk or negedge rst_n) begin

```

```

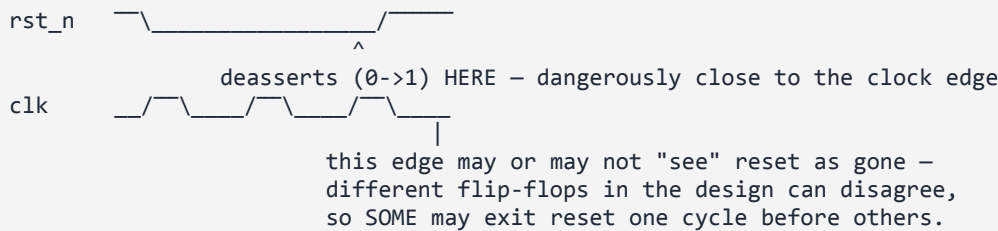
    if (!rst_n)
        q <= 1'b0;
    else
        q <= d;
end
Endmodule

```

■ COMMON PITFALL — Asynchronous Reset Deassertion Is a Timing Violation Waiting to Happen

Asserting an async reset is safe at any time — the flop is being forced to a known state. But deasserting it is different: rst_n going from 0 to 1 is itself an edge, and if that edge happens too close to an active clock edge, it can violate the recovery-time requirement of the flip-flop, causing the same metastability risk as a normal setup/hold violation (Section 4.3). This is why real designs synchronise the deassertion of an async reset before distributing it.

Seen as a timeline, the problem is a direct re-application of the setup/hold diagram from Section 4.4, but now the “data” signal is the reset deassertion edge itself:



This is exactly the reset synchroniser problem you are asked to solve and re-draw (correctly, aligned to clk) in Task 11 below.

9.3 Tasks

TASK 10: 4-bit Up-Down Counter with Asynchronous Reset

Implement a 4-bit up-down counter with an up_down control input and an active-low asynchronous reset rst_n. Reset must immediately clear the counter to 0, independent of the clock.

- Counter increments when up_down = 1 and decrements when up_down = 0, on every posedge clk (while rst_n is high).
- Testbench must: assert reset mid-count and confirm the counter clears immediately (not on the next clock edge); then release reset and verify both count-up and count-down sequences, including wraparound at 0 and 15.

TASK 11: Reset Synchroniser

Implement a 2-flop reset synchroniser that takes an asynchronous active-low reset (rst_n_async) and produces a synchronised active-low reset (rst_n_sync) that asserts immediately (asynchronously) but deasserts only on a clock edge, through two cascaded flip-flops. (Note: In case of active low, assertion is the transition from 1->0 while deassertion is from 0->1)

- Draw a timing diagram showing the problem: an unsynchronised async reset deasserting at a moment that violates the recovery time of a downstream flip-flop, and the resulting metastable/uncertain output.
- Draw a second timing diagram showing your synchroniser: the async assertion path (immediate) and the synchronised deassertion path (aligned to a clock edge, with the two-flop delay clearly shown), and explain how this removes the timing violation.

- Implement the synchroniser in SystemVerilog and testbench it by asserting/deasserting `rst_n_async` at various times relative to `clk`, including right at a clock edge, and showing on the waveform that `rst_n_sync` always deasserts cleanly aligned to `clk`.

“ See the Report Submission Format (Section 13) for how work on this topic must be documented and submitted. ”

10. Counters & Timing Applications

Goal: Apply flip-flops and reset logic to build practical timing circuits — a custom-modulus frequency divider and a full multi-digit stopwatch — and practice reasoning about frequency, duty cycle, and pulse width from a counter design.

10.1 Frequency Dividers

A modulo-N counter built from flip-flops divides an input clock frequency by N: it counts from 0 to N–1 and wraps, with a toggle or terminal-count output changing state once per full cycle of N input clock periods. T flip-flops are a natural building block here because a toggle is exactly “flip on this condition.”

Before dividing, look at the simplest possible counter — a free-running up counter with synchronous reset — as a base pattern you will extend for every task in this section:

```
module up_counter #(parameter WIDTH = 4) (
    input logic      clk, rst_n, en,
    output logic [WIDTH-1:0] count
);
    always_ff @(posedge clk) begin
        if (!rst_n)
            count <= '0;                // reset to all zeros
        else if (en)
            count <= count + 1'b1;      // free-running: wraps automatically
                                         // at 2^WIDTH due to fixed-width overflow
    end
endmodule
```

10.1.1 The Clock-Division Math

The relationship between an input clock frequency f_{in} , the divisor N you want, and the resulting output frequency f_{out} is simply $f_{out} = f_{in} / N$. Worked example: if your EDA Playground testbench clock runs at 100 MHz (period 10 ns) and you need a 10 MHz output (a divide-by-10), then $N = f_{in} / f_{out} = 100 \text{ MHz} / 10 \text{ MHz} = 10$, so you need a counter that counts 0..9 (10 states) and toggles or pulses its output once per full 10-count cycle.

■ KEY CONCEPT — Divide-by-N Needs $\text{ceil}(\log_2(N))$ Bits of Counter, Not N Bits

A common beginner mistake is assuming a divide-by-10 needs 10 flip-flops. It does not: a 4-bit counter can already count 0 through 15, more than enough to count 0 through 9 — you simply detect the terminal count (`count == 9`) and reset back to 0 on the next edge instead of letting it run to 15. This terminal-count-detect-and-wrap pattern is exactly what the modulo-10 divider task below, and the stopwatch's digit counters, both need.

10.2 Tasks

TASK 12: Modulo-10 Frequency Divider Using T Flip-Flops

Design a modulo-10 frequency divider using T flip-flops that divides the input clock frequency by 10.

- Write the SystemVerilog code for the divider (a 4-bit counter that resets to 0 after reaching 9, with an output that toggles or pulses once per 10 input clock cycles).
- Write a self-checking testbench that drives a known-period clock, counts the number of input clock edges between two edges of the divided output, and confirms the ratio is exactly 10.

TASK 13: 4-Digit Stopwatch

Design a stopwatch using counters that satisfies the following specification:

- It keeps time for up to 10 minutes, format M:SS:D (1 minute digit, 2 second digits, 1 tenths-of-a-second digit), counting from 0:00:0 to 9:59:9.
- It has a clear input clr; when asserted, the display resets to 0:00:0.
- The rightmost digit (tenths) increments every 0.1 seconds; when it rolls over from 9 to 0, the seconds digits increment.
- When the seconds reach 59 and roll over, the minute digit increments.
- Calculate the required internal counter frequency, duty cycle, and pulse width needed to generate the 0.1 s tick from your simulation clock, and show the calculation in your report.
- Write the SystemVerilog design (a cascade of modulo-10 / modulo-6 / modulo-10 digit counters, each enabled by the terminal count of the previous stage) and a testbench that runs the stopwatch through at least one full minute rollover and one full 10-tenths rollover, confirming digit values at each rollover boundary.

“ See the Report Submission Format (Section 13) for how work on this topic must be documented and submitted. ”

11. Finite State Machines

Goal: Learn to go from a word specification to a state diagram, to a state table, to working SystemVerilog — for both Moore and Mealy machines — and build a multi-input/multi-output controller of the kind you will reuse directly in the FIFO controller task in Section 12.

11.1 Moore vs Mealy

Machine	Output Depends On	Typical Tradeoff
Moore	Current state only	Output changes only on a clock edge (glitch-free), but may need one extra state/cycle compared to an equivalent Mealy machine
Mealy	Current state AND current inputs	Can react in the same cycle the input changes (fewer states), but the output can glitch if inputs are not clean

11.2 The Standard 3-Block FSM Coding Style

Every FSM in this module should be coded as three separate always blocks: (1) an always_ff state register that updates `current_state <= next_state` on the clock edge, (2) an always_comb next-state logic block that computes `next_state` from `current_state` and inputs, and (3) an always_comb (Moore) or combinational (Mealy) output logic block. Keeping these three blocks separate makes FSM code far easier to debug and to modify.

11.3 Worked Example: Detecting the Sequence “101” (Moore, with Overlap)

This full example walks through every step you will repeat for the tasks in Section 11.5 — from a plain-English specification all the way to working SystemVerilog. Read it carefully before starting Task 14.

Step 1: The Specification

Design a Moore FSM with one serial input bit in and one output detect. The circuit watches the incoming bit stream (one bit per clock) and must assert detect = 1 for one clock cycle whenever the most recently received 3 bits were 1, 0, 1 (in that order), including overlapping occurrences — e.g. the stream 1 0 1 0 1 must detect a match at both the first 101 and the overlapping 101 that starts at its third bit.

Step 2: The State Diagram (Described in Words and a Table)

A Moore machine's output depends only on which state it is in, so we need one state per “how much of the pattern has been seen so far”, plus a distinct state whose output is 1 once the full pattern has just been completed. Four states are enough:

State	Meaning	Output (detect)
S0	No useful prefix of “101” seen yet (reset state)	0
S1	Last bit seen was 1 (a possible start of the pattern)	0
S2	Last two bits seen were 1, 0	0
S3	Last three bits seen were 1, 0, 1 — pattern just completed	1

The transitions (what happens in each state depending on the next input bit) are:

Current State	in = 0	in = 1
S0	S0 (still nothing useful)	S1 (saw a 1: possible start)
S1	S2 (saw 1,0: keep going)	S1 (saw another 1: still just a possible start)
S2	S0 (saw 1,0,0: pattern broken, restart)	S3 (saw 1,0,1: MATCH!)
S3	S2 (overlap! The last 2 bits of this window were 0, could start 1,0,...)	S1 (overlap! last bit was 1, could start a new match)

■ KEY CONCEPT — Why S3's Transitions Look Unusual — That Is the Overlap Requirement

From S3 (just matched “101”), the next transition must NOT simply go back to S0, because the last one or two bits of the just-completed match might already be the start of a new match. From S3, on in=1 we go to S1 (not S0), because the bit we just matched on (the trailing “1” of “101”) is itself a valid “start of a new 101” — exactly the S0-to-S1 transition. This is precisely how the overlapping match in 1 0 1 0 1 is detected correctly.

Step 3: The SystemVerilog Implementation (3-Block Style)

```

module seq_detect_101 (
    input logic clk, rst_n, in,
    output logic detect
);
    typedef enum logic [1:0] {S0, S1, S2, S3} state_t;
    state_t current_state, next_state;

    // ---- Block 1: state register (sequential) ----
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            current_state <= S0;
        else
            current_state <= next_state;
    end

```

```

end

// ---- Block 2: next-state logic (combinational) ----
always_comb begin
    case (current_state)
        S0: next_state = in ? S1 : S0;
        S1: next_state = in ? S1 : S2;
        S2: next_state = in ? S3 : S0;
        S3: next_state = in ? S1 : S2;
        default: next_state = S0;
    endcase
end

// ---- Block 3: output logic (combinational, Moore: depends on state only) ----
always_comb begin
    detect = (current_state == S3);
end
endmodule

```

■ KEY CONCEPT — Why enum Instead of Raw Bit Patterns for the State Variable

typedef enum logic [1:0] {S0, S1, S2, S3} state_t; gives each state a readable name instead of forcing you to remember “2'b10 means S2” throughout the code. It also lets the simulator display current_state as S0, S1, etc. directly in the waveform viewer, which makes debugging an FSM dramatically faster — use this pattern for every FSM you write in this module, including the tasks below and the FIFO controller in Section 12.

Step 4: A Sketch of the Self-Checking Testbench

```

module tb;
    logic clk = 0, rst_n, in, detect;
    always #5 clk = ~clk;          // 100 MHz test clock

    seq_detect_101 dut (.clk(clk), .rst_n(rst_n), .in(in), .detect(detect));

    task automatic send_bit(input logic b, input logic expect_detect);
        in = b;
        @(posedge clk); #1;        // sample just after the edge (Section 6.3)
        if (detect != expect_detect)
            $display("[FAIL] bit=%0b detect=%0b expected=%0b", b, detect, expect_detect);
        else
            $display("[PASS] bit=%0b detect=%0b", b, detect);
    endtask

    initial begin
        rst_n = 0; in = 0; @(posedge clk); rst_n = 1;
        // stream: 1 0 1 0 1 -> expect detect=1 after bit 3 (index2) AND after bit 5 (index4,
        // overlap)
        send_bit(1, 0);
        send_bit(0, 0);
        send_bit(1, 1); // first 1,0,1 complete
        send_bit(0, 0);
        send_bit(1, 1); // overlapping 1,0,1 complete
        $finish;
    end
endmodule

```

■ DESIGN NOTE — Multi-Input/Multi-Output FSMs

A multi-input, multi-output FSM is simply an FSM whose next-state logic depends on more than one input signal and whose output logic drives more than one output signal — exactly the coding style above, just with wider input/output

vectors in the sensitivity of each block. The read-modify-write pattern (read a value, transform it, write it back) that you may have seen described elsewhere is not practiced as a separate exercise here — you will apply it directly, for real, when you build the FIFO controller's pointer and status logic in Section 12.

11.4 Excitation Tables: Deriving Flip-Flop Inputs From a State Sequence

The worked example above (Section 11.3) used the modern coding style: an enum state register plus a case statement, which the synthesis tool turns into flip-flops and gates for you. The first task below (the custom-sequence counter) instead asks you to design directly at the flip-flop level using T flip-flops — the classical way FSMs were designed before enum-based coding became standard, and a skill you need whenever you must reason about exactly which flip-flop inputs produce a given transition.

An excitation table answers the question: “given that a flip-flop currently holds Q and must next hold Q_{next} , what input value does it need?” For a T flip-flop (Section 9.1: $Q_{\text{next}} = Q \text{ XOR } T$), the answer follows directly from that equation:

Current Q	Next Q_{next}	Required T
0	0	0 (stay at 0: no toggle)
0	1	1 (must toggle to reach 1)
1	0	1 (must toggle to reach 0)
1	1	0 (stay at 1: no toggle)

Worked mini-example: suppose a single T flip-flop must follow the repeating sequence $Q = 0, 1, 0, 1, \dots$ (i.e. it simply toggles every cycle). Reading each $Q \rightarrow Q_{\text{next}}$ pair against the table above: $0 \rightarrow 1$ needs $T=1$, and $1 \rightarrow 0$ needs $T=1$ — so T must be tied to a constant 1, which matches the toggle behaviour you'd expect. For a counter with several flip-flops, you build one such table PER flip-flop, reading off the required T value for every state transition in your state diagram, and the resulting T equations (as a function of the current state bits) become your next-state logic — exactly what Task 14 below asks you to do for a 5-state sequence.

11.5 Tasks

TASK 14: Custom-Sequence Counter with T Flip-Flops (State Machine View)

Design a counter that repeats the binary sequence 0, 1, 3, 4, 7 (and then back to 0), implemented using T flip-flops.

- Draw the state diagram showing the 5 states and their transitions.
- Derive the state table, including the T flip-flop excitation values needed for each transition (use the method and worked mini-example in Section 11.4).
- Draw the final gate-level circuit (T flip-flops plus next-state logic) and implement it in SystemVerilog.
- Testbench must run the counter through at least 3 full cycles and confirm it always follows $0 \rightarrow 1 \rightarrow 3 \rightarrow 4 \rightarrow 7 \rightarrow 0 \dots$ and never enters an unused state (2, 5, or 6).

TASK 15: Multi-Input, Multi-Output FSM

Design a Moore FSM with at least two independent input signals (e.g. start, cancel) and at least two independent output signals (e.g. busy, done) that models a simple request-processing controller: it waits in IDLE, moves to BUSY when start is asserted, asserts busy while processing for a fixed number of cycles, asserts done for one cycle on completion, and returns to IDLE — unless cancel is asserted while BUSY, in which case it aborts back to IDLE without asserting done.

- Draw the full state diagram labelling every transition condition and every state's output values.
- Implement using the 3-block coding style from Section 11.2.

- Testbench must cover: a normal start→busy→done→idle sequence, and a start→busy→cancel→idle sequence.

TASK 16: Mealy FSM: Serial Binary Input Divisible by 3

Design a Mealy state machine that receives a serial binary input (MSB first, one bit per clock) and outputs 1 whenever the binary number seen so far (as a running value) is exactly divisible by 3, and 0 otherwise. Hint: track the running remainder mod 3 as your state.

- Draw the state transition diagram (3 states, one per possible remainder).
- Develop the next-state and output logic equations from the state table.
- Write the SystemVerilog code for the FSM and a testbench that feeds in several known binary sequences (e.g. 110 = 6, which is divisible by 3) and confirms the output asserts at the correct bit position.

“ See the Report Submission Format (Section 13) for how work on this topic must be documented and submitted. ”

12. Synchronous FIFO Controller

This is the final task of the module, and it is deliberately the largest single design in it — by this point you have every tool you need: data types and operators (Section 4), combinational design (Sections 6–8), sequential design and reset (Section 9), counters (Section 10), and FSMs (Section 11). It is not re-taught topic by topic here; instead it asks you to combine everything above into one realistic, parameterized hardware block, submitted in the same report format as every other section.

12.1 Parameterized Modules (New Syntax You Will Need Here)

Every module so far in this document has had fixed port widths (e.g. always exactly 4 bits). A parameterized module instead declares one or more parameters in a `#( ... )` list right after the module name, which let the SAME module be instantiated with different widths or depths without rewriting any code — exactly like a function template. Parameters are given default values, which are used unless the instantiating code overrides them.

```
module register_generic #(
    parameter int WIDTH = 8          // default width if not overridden
)(
    input  logic          clk, en,
    input  logic [WIDTH-1:0] d,
    output logic [WIDTH-1:0] q
);
    always_ff @(posedge clk)
        if (en) q <= d;
endmodule

// Instantiating with the default WIDTH=8:
register_generic u_reg8 (.clk(clk), .en(en), .d(data8), .q(out8));

// Instantiating with an OVERRIDDEN parameter, WIDTH=16:
register_generic #(.WIDTH(16)) u_reg16 (.clk(clk), .en(en), .d(data16), .q(out16));
```

■ KEY CONCEPT — Why the FIFO Task Needs This

The FIFO interface below is declared with parameters `ADDR_WIDTH` and `DATA_WIDTH`, exactly like `WIDTH` above. Design and testbench against the default values (`ADDR_WIDTH=3`, i.e. a depth-8 FIFO) — you do not need to test multiple parameter values, you only need to write the RTL so that it WOULD work correctly if someone else

instantiated it with different values later. That is what “fully parameterized” means in the specification below: no hardcoded 8 or 3 anywhere in your RTL — use the parameter names throughout.

12.2 Objective

Design, implement, and verify a synchronous FIFO (First-In-First-Out) memory controller with an active-low asynchronous reset, following the specification below. This block is a realistic combination of a counter-based read/write pointer datapath, comparison logic for full/empty detection, and a small FSM-style control path — the same category of design you will keep encountering for the rest of the programme.

12.3 Specification

- Default depth of 8 memory locations, each storing 8-bit data; fully parameterized with ADDR_WIDTH (depth = $2^{\text{ADDR_WIDTH}}$) and DATA_WIDTH.
- Separate read and write ports; simultaneous read and write operations must be supported.
- Data must be read back in the exact order it was written (true FIFO ordering).
- Flow control: reads are blocked (rd_valid stays low) when empty; writes are blocked (wr_ready stays low) when full.
- Status signals: at least full, empty, and count.
- Edge cases: a read attempted on an empty FIFO must not corrupt stored data; a write attempted on a full FIFO must not overwrite existing data.

12.4 Module Interface

```
module fifo_controller #(
    parameter int ADDR_WIDTH = 3,    // default depth = 8
    parameter int DATA_WIDTH = 8
)(
    input logic          clk,
    input logic          rst_n,      // asynchronous active-low reset

    // Write port
    input logic          wr_en,
    input logic [DATA_WIDTH-1:0] wr_data,
    output logic         wr_ready,

    // Read port
    input logic          rd_en,
    output logic [DATA_WIDTH-1:0] rd_data,
    output logic         rd_valid,

    // Status
    output logic         full,
    output logic         empty,
    output logic [ADDR_WIDTH:0] count
);
```

12.5 Suggested Design Approach

- Start with the read and write pointers as two separate modulo- $2^{\text{ADDR_WIDTH}}$ counters (reuse your Section 10 counter pattern), each incrementing on its own enable condition.
- Derive full and empty from comparing the pointers (a common technique: extend each pointer by one extra bit so full and empty can be distinguished even though both conditions mean “pointers equal” in the lower bits).

- Use `always_ff` for the pointer registers and the memory array, and `always_comb` for `wr_ready`, `rd_valid`, `full`, `empty`, and `count`.
- Treat `rst_n` exactly as in Section 9: asynchronous assertion, and pass it through a reset synchroniser if you want to demonstrate best practice (optional, note it clearly in your report if you do).

12.6 Tasks

TASK 17: Synchronous FIFO Controller

Implement the `fifo_controller` module to the specification and interface above.

- RTL must be built from the blocks covered in this module: a read/write pointer pair (Section 9–10 style counters, with correct reset behaviour per Section 9), comparison logic for full/empty (combinational design, Section 6), and any explicit state-tracking you choose to add as an FSM (Section 11).
- A self-checking testbench (Section 5.3 style) must verify: reset behaviour, normal read/write operation, full and empty conditions, simultaneous read/write behaviour, and overflow/underflow handling without data corruption.

TASK 18: Extension: Almost-Full / Almost-Empty Flags

Extend Task 17's FIFO with two additional status outputs, `almost_full` and `almost_empty`, that assert when count is within 1 of full or empty respectively (i.e. one write away from full, or one read away from empty).

- This is a common real-world addition used to let an upstream producer start slowing down before the FIFO is actually full, rather than reacting only once it already is.
- Testbench must drive the FIFO to depth–1 items and confirm `almost_full` asserts (and `full` does not), then to 1 item and confirm `almost_empty` asserts (and `empty` does not).

“ See the Report Submission Format (Section 13) for how work on this topic must be documented and submitted. ”

13. Report Submission Format

Every topic section in this module (Sections 5–12) requires one PDF report submitted separately of at least one task, there is no separate submission process for it. The report must contain the following, in this order, for every task in that section:

#	Required Item	Notes
1	EDA Playground link	Placed at the top of the design and testbench code for that task; the link must be shareable (set to public/unlisted, not private).
2	Schematic diagram	A gate-level diagram (draw.io) for small tasks; a top-level block diagram (draw.io) for lengthier / multi-module tasks such as the barrel shifter, stopwatch, FSMs, and the FIFO controller.
3	Terminal output screenshot	A screenshot of the simulation log showing the results printed by your testbench (e.g. [PASS]/[FAIL] lines or \$display output).
4	Waveform screenshot	Required wherever the task involves sequential behaviour, timing, or multiple stimulus vectors over time (all of Sections 9–12, and any Section 6–8 task where waveform inspection adds value).
5	Explanation	A short written explanation wherever the task explicitly asks for one (e.g. Boolean derivations, timing diagrams, logic-level/critical-path reasoning, Moore vs Mealy justification).

■ DESIGN NOTE — File Naming and Structure

Name each report Module3_Section<N>_<YourName>.pdf. The Section 12 report follows the exact same 5-item format as every other section above — there is no separate repository requirement, though you are welcome to also share a GitHub link alongside your EDA Playground link if you would like the extra practice with version control from Module 1.

■ COMMON PITFALL — What Gets a Report Bounced Back

A missing or private EDA Playground link, a screenshot of code instead of simulation output, a schematic that does not match the submitted RTL, or a written explanation that restates the question instead of answering it, are the most common reasons a report is sent back for resubmission. Check all four/five items are present before submitting.

14. Recommended Resources

Primary References

- ChipVerify SystemVerilog Tutorial — <https://chipverify.com/tutorials/systemverilog>
- VLSIVerify SystemVerilog Guide — <https://vlsiverify.com/systemverilog/>
- EDA Playground — <https://edaplayground.com>
- draw.io (diagrams.net) — <https://draw.io>

Quick Reference / Cheat Sheets

- Doulos SystemVerilog for Verification / Golden Reference Guide — <https://www.doulos.com/knowhow/systemverilog/>
- ASIC-World Verilog Tutorial — <https://www.asic-world.com/verilog/veritut.html>
- Nandland Verilog & SystemVerilog Tutorials (FPGA-focused, beginner-friendly) — <https://nandland.com/verilog-and-systemverilog-tutorials/>

Coding Style: Why This Module Teaches the Rules It Does

Sections 6.2 (blocking vs non-blocking) and 9.2 -- 9.3 (reset style) are not arbitrary rules — they follow industry-standard guidelines first published by Clifford E. Cummings and still cited as the reference on these exact topics. Reading the originals is optional but strongly encouraged if a rule in this module feels unmotivated:

- Cummings — “Nonblocking Assignments in Verilog Synthesis” (SNUG 2000) — http://www.sunburst-design.com/papers/CummingsSNUG2000SJ_NBA.pdf
- Cummings — “Reset— What's a Simple Thing Like Reset Have To Do With Synthesis?” (SNUG 2003) — http://www.sunburst-design.com/papers/CummingsSNUG2003Boston_Resets.pdf

Digital Logic Background

- Digital Design, M. Morris Mano — chapters on combinational logic, sequential logic, and FSM design (state diagrams, state tables, excitation tables).
- Any standard Boolean algebra / Karnaugh map reference for the minimization steps required in Section 6's tasks.

Practice

- HDLBits — bite-sized SystemVerilog exercises covering Sections 6–11 — <https://hdlbits.01xz.net>

15. Glossary of Terms

A quick reference for every technical term used in this module, in the order the concept is first needed. If you meet a word you don't recognise anywhere in Sections 4–12, it is almost certainly defined here.

Term	Definition
Module	A named, reusable block of hardware with defined input/output ports (Section 4.0).
Port	An input, output, or inout connection point on a module.
Instantiation	Placing one module inside another, wiring its ports to local signals (Section 4.0.2).
Testbench	A SystemVerilog module written purely to generate stimulus for, and check the output of, a design under test; never synthesized into real hardware.
DUT	Device Under Test — the design module instantiated and driven by a testbench.
4-state logic	A value system with 0, 1, X (unknown), and Z (high-impedance/floating) (Section 4.1).
X (unknown)	A signal value the simulator cannot determine — undriven, or driven to conflicting values.
Z (high-impedance)	A signal that is not currently being driven by anything (floating).
Combinational logic	Logic whose output depends only on its current inputs, with no memory of the past (Section 4.3).
Sequential logic	Logic whose output depends on stored state and only changes on a clock edge (Section 4.3).
always_comb	A procedural block for describing combinational logic; SystemVerilog infers its sensitivity list automatically.
always_ff	A procedural block for describing sequential (clocked) logic, e.g. flip-flops and registers.
Blocking assignment (=)	Executes immediately, in program order; used inside always_comb (Section 6.2).
Non-blocking assignment (<=)	Schedules an update that applies to all such assignments together at the end of the time step; used inside always_ff (Section 6.2).
Latch	An unintended (or intended) level-sensitive memory element, most often created by mistake via an incomplete always_comb (Section 6.1).
Setup time (t _{su})	The minimum time data must be stable before a clock edge for a flip-flop to reliably capture it (Section 4.4).
Hold time (t _h)	The minimum time data must remain stable after a clock edge for a flip-flop to reliably capture it (Section 4.4).
Metastability	An unstable, unpredictable output state a flip-flop can enter when setup/hold requirements are violated (Section 4.4).
Decoder	A circuit that asserts exactly one of 2 ⁿ outputs based on an n-bit binary input code (Section 7.1).
Encoder / Priority Encoder	A circuit that converts multiple asserted input lines into the binary index of the (highest-priority) asserted line (Section 7.2).
One-hot encoding	An encoding scheme where exactly one bit of a wide vector is ever 1 at a time (Section 7.2).
Multiplexer (MUX)	A circuit that selects one of several data inputs to route to its output, based on a select input (Section 8.1).

Term	Definition
Barrel shifter	A circuit that shifts or rotates a vector by a run-time-selectable amount, typically built from cascaded muxes (Section 8.2).
Flip-flop	A one-bit memory element that captures its input only on an active clock edge and holds it otherwise (Section 9.1).
Register	A group of flip-flops sharing a common clock (and usually a common enable), storing a multi-bit value (Section 9.1).
Synchronous reset	A reset that only takes effect on the next active clock edge after being asserted (Section 9.2).
Asynchronous reset	A reset that takes effect immediately, independent of the clock (Section 9.2).
Reset synchroniser	A small circuit that lets an asynchronous reset assert immediately but forces its deassertion to align to a clock edge, avoiding a timing violation (Section 9.3).
Counter	A register that advances through a defined sequence of values, usually by +1 each active clock edge (Section 10.1).
Terminal count	The specific counter value at which a modulo-N counter must wrap back to its starting value (Section 10.1.1).
Finite State Machine (FSM)	A sequential circuit whose behaviour is described by a finite set of named states, and rules for moving between them (Section 11).
Moore machine	An FSM whose outputs depend only on the current state (Section 11.1).
Mealy machine	An FSM whose outputs depend on the current state AND the current inputs (Section 11.1).
State register	The always_ff block in the 3-block FSM style that stores the FSM's current state (Section 11.2).
Next-state logic	The combinational block in the 3-block FSM style that computes what state to move to (Section 11.2).
FIFO	First-In-First-Out memory: data is read back in exactly the order it was written (Section 12).
RTL	Register-Transfer Level — the level of abstraction (registers plus the combinational logic that transfers data between them) at which this entire module is written.
EPWave	The waveform viewer built into EDA Playground, used to visually inspect signal values over time (Section 5.2.2).
VCD	Value Change Dump — the file format (\$dumpfile / \$dumpvars) that records signal transitions for viewing in EPWave.

End of Module 3

MEDS Lab • UET Lahore • Summer 2026

“If C is the hardware in text form, SystemVerilog is the hardware in circuit form.”